

编写导航架构指南

模型： google/gemma-4-e4b
创建时间： 2026年4月27日下午6:11:36

用户

请介绍一下 Kotlin Compose 中的导航功能。

助手

当然！ Jetpack Compose 中的导航是最重要、最常用的主题之一,也是最具挑战性的主题之一,因为它需要转变思维方式:我们从命令式方法（Activity A 调用 Activity B)转变为声明式方法（UI 取决于当前状态）。

如果我们谈论 “专业”导航,那么它不仅仅是调用 `NavController.navigate( route )`,而是通过导航图管理应用程序状态的架构方法,以及对MVVM/MVI 最佳实践的遵守。

以下是对关键概念、工具和高级模式的详细概述。

基本工具（基础知识）

这一切都始于 `androidx.navigation: navigation-compose` 库。

1.导航控制器

这是主导航控制器。它负责跟踪当前状态、管理屏幕堆栈并执行过渡效果。

```
// 获取 Composable 作用域中的控制器实例 val navController = rememberNavController()
```

2.导航主机

这是宿主组件,它知道所有可能的路由 (Routes) ,并根据当前状态的navController 显示相应的 Composable。

```
@Composable
fun AppNavHost(navController: NavHostController) {
    NavHost(
        startDestination = home, navController = navController
    ) { backStackEntry ->
        composable( home ) { HomeScreen(onNavigateToDetails = { ...

        >composable( details/{itemId} ) { backStackEntry -> // 参数: получение ID из пути val itemId =
            backStackEntry.arguments?.getString( itemId )?:

            DetailScreen(itemId = itemId)
```

<高级概念 (该 “专业版” 等级)

达到专家级别后,你不应该考虑流量,而应该考虑应用程序的状态。

1.管理参数 (类型化参数)

与其将 ID 作为字符串传递并在每个屏幕上解析,不如使用结构化参数。组合导航允许您在路径中使用数据类型:

```
>// 1. Определение маршрута с типом параметра 可组合(
    路由 = product/{productId}/{category} , 参数 =
    listOf(navArgument( productId ) { 类型 =
NavType.StringType }) ) { backStackEntry
->
    // 2. Безопасное получение аргумента нужного типа val ProductId: String =
    backStackEntry.arguments?.getString( productId )?:
```

```
<ProductScreen(productId = productId)
```

```
> // 名称:
navController.navigate( product/$id/ electronics )
```

<2.使用路由器（中央调度器）

初学者最常犯的错误是让每个屏幕都知道如何跳转到下一个屏幕。正确的架构是:创建一个独立的调度器类（路由器），它负责所有流量转换,并了解所有可能的路由。

例如:不要将 onNavigateToDetails 回调传递给 HomeScreen:

```
> // Плохо: HomeScreen сам вызывает навигацию @Composable
fun
HomeScreen(navController: NavController) { Button(onClick =
    { navController.navigate( details/$id ) }) {}
```

使用路由器（中介者/协调器模式）:

创建一个 AppRouter 或从了解整体应用上下文的父组件传递一个处理函数,而不是 NavController。这样可以使屏幕尽可能独立于导航机制。

3.回退栈管理

有时,当用户返回时,不仅需要返回到上一个屏幕,还需要保存一些数据或执行一些操作（例如,更新列表）。

popUpTo:允许您从堆栈中“卸载”多个屏幕。

```
// 跳转到个人资料页面,完全从堆栈中移除主屏幕 navController.navigate( profile )
{
    popUpTo( home ){ inclusive = true } // 删除所有到 “home”的元素
```

• launchSingleTop:确保如果您尝试导航到堆栈中已存在的屏幕,则不会创建此屏幕的新副本。

4.深度链接

这对于与外部系统（例如 Brauseyandex、 Direct、推送通知）集成至关重要。 “创建导航”功能允许您将 URL 链接到路线上的特定位置：

- 1.在nav_graph.xml或代码中 :声明链接方案。
- 2. NavHost 中的处理 :导航会自动处理传入的链接并导航到所需的屏幕 ,就像用户在应用程序中点击按钮一样。

5.国家恢复

当您离开屏幕（屏幕A）然后从屏幕B 返回到屏幕 A 时,数据可能会丢失。

决定：

- 1.在 ViewModel 中保存状态 :移动所有关键任务项
ViewModel 中的数据与此 Composable 的生命周期绑定。即使 Compose 销毁并恢复了 Composable， ViewModel 仍然存活。
- 2.使用 StateFlow / LiveData :使用响应式流将数据从业务逻辑 (ViewModel)直接传输到用户界面 ,无需依赖导航参数。



汇总表： “专业人士”与 “初学者”

方面	初学者入门指南 （初学者）	专业态度 （专业/建筑）
广播数据源	路径中的 Steam 参数（details/{id}） ,原始参数。	使用 ViewModel 作为数据源。参数 仅用于 ID。
管理点击率	每个可组合对象都会调用导航本身（代码混乱）。	中央路由器/协调器,它封装了所有逻辑点击操作,并且知道所有路由。
屏幕状况	仅依赖于参数 backStackEntry。	使用 ViewModel 保存状态 ,而不管 Composable 循环的生命周期如何。
加工 错误	不进行任何处理或采用最简单的try/catch 方法。	实现集中式系统错误处理（例如,结果包装器)和路径处理程序。

方面	初学者入门指南 (初学者)	专业态度 (专业/建筑)
测试	导航功能难以测试。	导航图和路由很容易理解,因为它们很 容易被赋予单元测试,因为点击逻辑被放置在类 中。